



content-engine

GUIDE

The Ladder of Abstraction

How to tell which AI work is about to be commoditized, and where to invest your time so it is not gone in six months.

content-engine (Will Vessels) · 2026-06-27

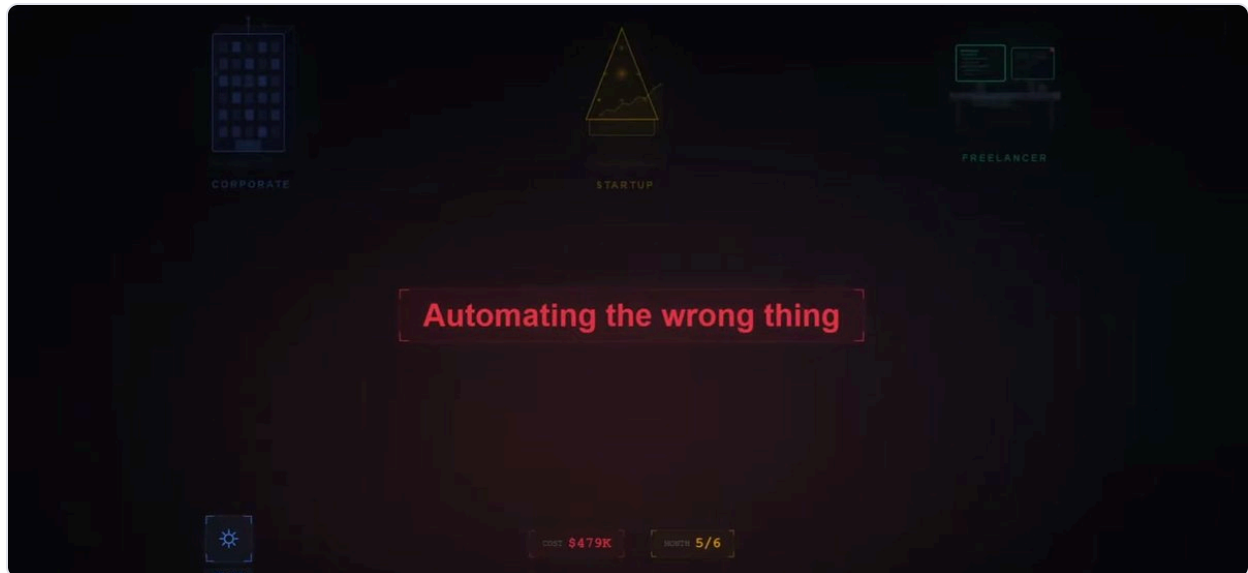
For: Builders, freelancers, and teams deciding what to automate with AI

Learning objectives

- Locate where any piece of your work sits on the ladder of abstraction (book, movie, or game).
- Decide what to give away for free and what to actually charge for.
- Avoid automating work that the next AI model will hand you for free.
- Name your concrete next move instead of generic adapt-to-AI advice.

The trap: automating the wrong thing

The same mistake shows up across corporate sessions, startups, and freelancers: people automate the wrong thing. They spend six months building an internal tool, training the team on it, and rolling it out. Then the next AI model update drops, the capability ships built-in and free, and any moat they thought they had is gone.



The trap: pouring months and budget into automating the wrong layer.

One Fortune 500 team spent roughly \$200,000 in real money and engineering time on a custom AI workflow for document processing. Then the platform they were feeding documents into added that exact process natively: one API call, a single button. Months of work became a toggle in someone else's settings menu.

WARNING

The trap is not AI replacing your job. The trap is investing in automating something the next model was always going to handle on its own anyway.

The core idea: layers of abstraction

It starts with a story: characters, conflict, resolution. As a **book**, that story is text on a page, the same experience for every reader, and you can print a million identical copies. Make the same story a **movie** and hundreds of millions of dollars go into actors, effects, sound, and a

production team. The narrative underneath is the same, but the experience is different and it costs far more to produce, because the layer of abstraction is thicker.

Make that same story a **video game** and production runs into the billions. Here every player's experience is different: you make choices, you control the pacing, and the story responds to you. Your playthrough is not the same as mine. Same story, three levels of abstraction. Each layer adds more involvement and control, and each one captures exponentially more value, not because the story got better, but because the experience got harder to replicate.

NOTE

A new layer of abstraction does not kill what is underneath. It changes what that layer is worth. Books did not die when movies arrived; people still make and lose money on books. But anyone selling only the raw story, plain information with nothing wrapped around it, had and still has a problem.

Why this applies to software and AI

Software works the same way, and this is where most people get it wrong. They hear that AI can build a website and assume the website is the product. The website was never the product; it was always the book version of something deeper, the raw output that anyone can produce.

The actual value in any business lives in the process underneath: how the pieces connect, which parts need a human, which parts a tool handles better, and which parts you leave alone entirely. That is the real work, and it is what most people are currently skipping at every abstraction layer.

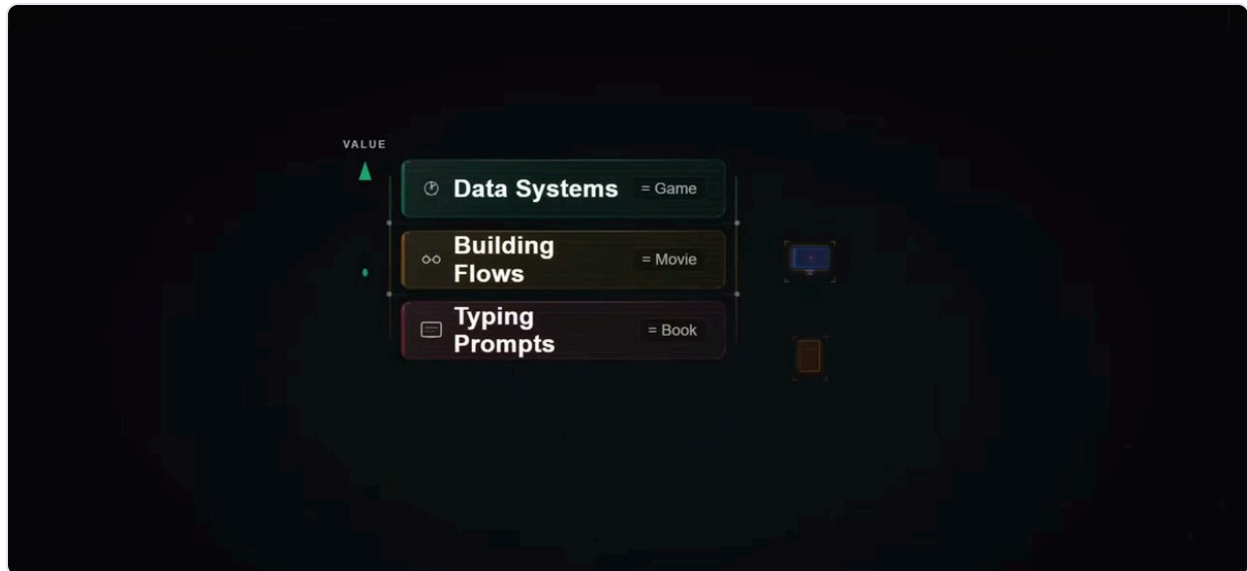
TIP

When the speaker works with a company, the first step is to break apart every workflow, not just engineering but go-to-market, HR, content, operations, and development, and decide task by task where AI actually helps, where traditional tooling beats AI, and where you just need a person. That breakdown alone is more valuable than people realize.

The ladder: three layers of AI work

Most people are at **layer 1**: typing prompts into a chat tool and getting outputs. At this level you are using AI like a book, the same tool, same input, and same results for everyone who has it.

Layer 2 is building flows: connecting prompts to processes and understanding which tasks in your specific business get automated and which do not. That is the movie, production value and context that take real knowledge to do well. **Layer 3** is when those flows generate data and you use that data to build systems that get smarter over time, bespoke, interactive, and responsive, more like a video game.



The ladder of abstraction: typing prompts (book), building flows (movie), data systems (game), rising in value.

NOTE

This is the same pattern as programming languages: machine code to assembly to C to Python. The people who thrived were not the ones who memorized the lower layer, but the ones who understood what the new layer made possible.

What to invest in (and what to give away)

The instinct right now is to automate everything you can: you see a slow process, throw AI at it, save time, and feel productive. The question almost nobody asks is whether the thing you just automated will be a built-in feature in six months. If it is, you spent your energy on something with a short expiration date.

1. For each task, ask: is this likely to be a built-in feature in six months? If yes, do not invest in automating it. Do not automate what the next model will give you for free.

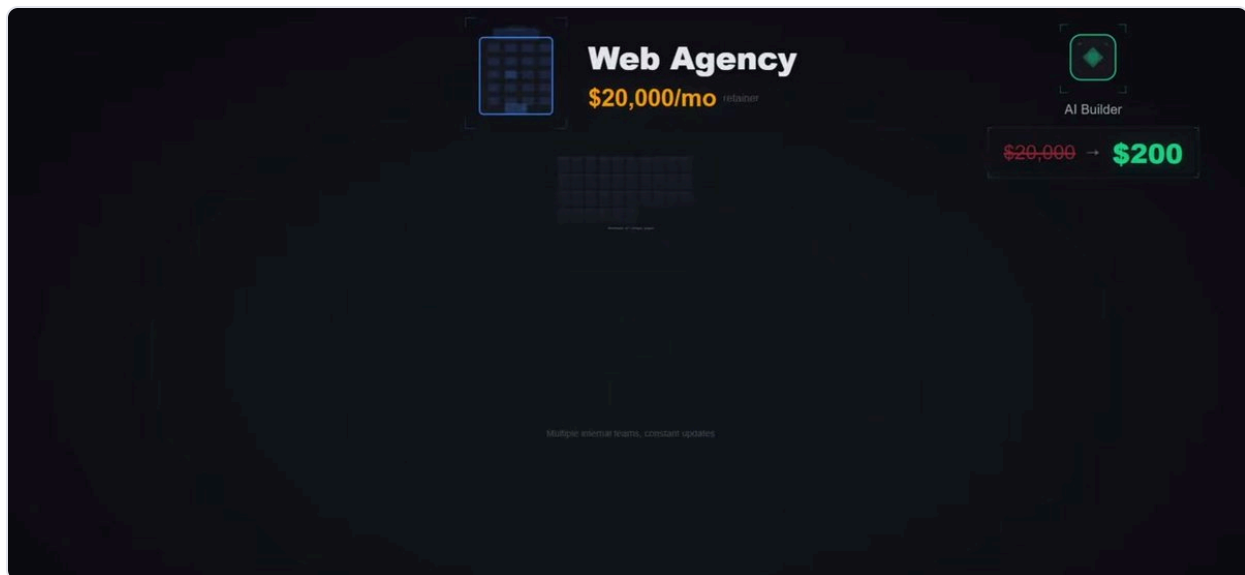
2. Invest instead in what is specific to your context: your domain knowledge, your client relationships, and your data, the things that cannot be scraped and fine-tuned into a general model because they only exist inside your situation.
3. Whatever AI made cheap, give it away. That is your book layer, the thing that gets people in the door.
4. Charge for the layer above: the involved, context-heavy work that takes judgment, taste, or real relationships.

WARNING

Automating a soon-to-be-native feature is building the tutorial level of a game that has not even released yet.

Worked example: the web agency that moved up a layer

A web agency used to charge about *\$20,000 a month* to maintain a large client base: hundreds of unique pages, multiple internal teams, and constant updates. A tool like Lovable can now generate pages, layouts, and basic HTML for a couple hundred dollars. That part is commoditized, the book layer.



The commoditized layer: a \$20,000/mo retainer's raw page output now costs about \$200 from an AI builder.

But this agency does more than write HTML. They interview the client's employees to understand internal knowledge structures and make design decisions based on user psychology and conversion data specific to that company's audience, things you cannot prompt because they came from years of being embedded in the business.

What they did was stop fighting commoditization and use it: they cut the cost of basic work dramatically and built a portal where the client's team handles simple updates themselves, work that used to need a support ticket and a two-day turnaround. Now the agency spends almost all its time on high-context work, the design strategy and research that require taste and judgment. They went from selling movies to selling video games: same client, same general service, but now the client is involved and the agency owns the bespoke layer.

The same rule in reverse: do not over-build

This works the other direction too. Companies spend a year building a full software product before they have even validated the idea. What they usually need is a 60% solution built with basic, free AI tools, run internally to figure out what matters, and only then a decision on whether the full build is worth it.

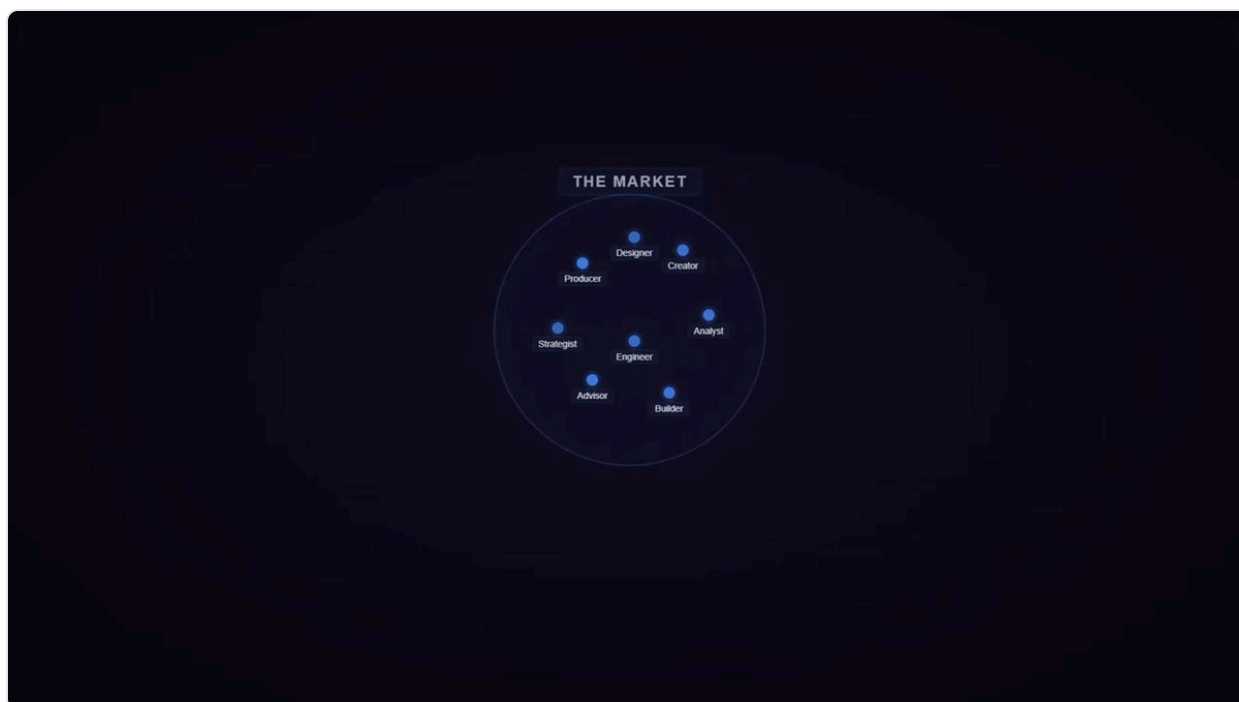
TIP

Use the book version while you test the story. Do not build the movie until you know people want to watch it.

Why the market itself survives

When assembly language arrived, the people writing machine code by hand did not all disappear; some moved up. And new problems showed up that only existed because assembly existed, problems nobody could have predicted, because you cannot see the next layer's problems from the current one. C abstracted assembly, Python abstracted C, frameworks abstracted Python, and now AI tools are abstracting the frameworks. Each time, the layer below became infrastructure and the new layer created entirely new categories of work.

There has never been a point in computing history where a new abstraction layer reduced the total amount of work humans do. Not once. Automation does not collapse markets; it kills specific rent-seeking positions within them, the middlemen who were only in business because friction protected them. The market itself is as durable as human desire: even if AI does your shopping, finds the best price, and handles payment, you are still the one who is hungry or wants a specific pair of shoes. The transaction chain still ends at a person with a preference.



The market is a web of durable human roles: producer, designer, creator, strategist, engineer, analyst, advisor, builder.

Your next move

Different industries sit at different points right now. Some are still selling books, raw features and commodity work, and are about to have a few rough years. Some are at the movie layer, real production value but the same experience for every client, and need to find the video-game version. A few are already selling games and are staring at the question of what is more bespoke than that. That frontier has no answer yet, because you can never see the next layer from this one, but it always shows up.

NOTE

The question was never whether AI will replace you. The question is: what am I selling, which layer am I standing on, and what does the layer up look like? Figure that out. Give away the movies; sell the video games.

Explains Every AI Failure' by Jake Van Clief (public YouTube). Attributed to Jake Van Clief.